Universal Pool Transformers: Lifted Sites Make Foundation Models Robust to Sensor Failure

Anonymous Author(s)
Affiliation
Address
email

Abstract

Virtual sensing predicts the reading of a target sensor from neighbouring sensors. Every deployment is different, labelled calibration data are scarce, and sensors fail. In-context learners such as TabPFN adapt to a new deployment from a few labelled rows, but their predictive distributions degrade sharply when input sensors go missing. We characterize the structure that missing sensors require. Products of per-sensor experts are the free construction on sensors, the only predictors whose natural parameters are additive over observation masks. Scalar experts cannot fuse sensors with shared noise, whereas sites *lifted* to a target-plus-nuisance latent are exact for factor-structured noise. The *Universal Pool Transformer* (UPT) puts a lifted-site head and an unconstrained head on one transformer backbone and combines them by the linear opinion pool, the pooling rule that commutes with marginalization. Its log-likelihood is never worse than either head's by more than the log of that head's weight. The same rule pools UPT with a foundation model, with weights chosen per episode by cross-fitting the episode's own labelled rows.

Under protocols committed before their data were scored, we tested this on four sensor networks with test periods in 2026, after the release of TabPFN v2. UPT $\oplus$ TabPFN improves TabPFN v2's log-likelihood on every network, by +0.17 [+0.14, +0.20] nats with all inputs and by +0.60 [+0.49, +0.72] nats with six inputs missing. It also improves TabPFN-3.5, a much stronger model released in September 2026, by +0.04 [+0.03, +0.06] and +0.03 [+0.01, +0.04] nats. UPT is a better partner than a matched-scale transformer (+0.12 [+0.08, +0.16]) or the closed-form sites (+0.10 [+0.08, +0.12]). Across thirteen networks in five domains and nine pre-registered protocols, UPT also beats matched-scale transformers, neural processes and kriging. We report every failure, including the networks on which UPT alone loses to TabPFN v2.

1 Introduction

Sensor networks now measure traffic, air quality, weather and rivers at a density that no maintenance budget can keep fully operational. *Virtual sensing*, or spatial kriging, predicts a sensor's reading from the readings of its neighbours. Its uses include filling gaps, standing in for failed or decommissioned instruments, and placing low-cost virtual sensors where no instrument exists [31, 5]. Three features make the problem hard.

1. **Deployments differ.** The relation between a target and its neighbours changes from site to site and over time, and only a short labelled record is available for each.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

2. **Noise is shared.** Weather, traffic waves and local emissions move several neighbours at once, so neighbours are not conditionally independent given the target.
3. **Sensors fail.** At prediction time an arbitrary subset of the neighbours is available, often a subset never seen together in the labelled record.

In-context virtual sensing. We cast the problem as in-context probabilistic prediction. A model receives a deployment's labelled rows (the support), with their missing entries, and a query made of an arbitrary subset of neighbour readings. It returns a predictive distribution for the target. Trained once, such a model adapts to a new deployment from its support alone, as neural processes [8, 25] and prior-fitted networks do [24, 16]. The pretrained TabPFN v2 [16] is a strong example. As we show, however, its predictive distribution degrades rapidly as input sensors go missing (§5.1).

Structure first. Missing sensors call for a predictor that is defined for every subset of inputs and handles unseen subsets gracefully.

- **The free construction.** Products of per-sensor experts meet this requirement by construction: a missing sensor contributes no factor [15, 30]. We show that this is a universal property, not a design choice. Natural-parameter additivity over masks characterizes products of sites, the free construction on sensors (Proposition 1).
- **Scalar experts are not enough.** With shared noise, no product of mask-independent experts over the target alone is Bayes-consistent (Proposition 2). Sites *lifted* to a latent over target and shared nuisance are exact, and need as many nuisance dimensions as the rank of the shared noise (Propositions 3–3).
- **Combining with a flexible model.** Real sensors are nonlinear, which favours unconstrained transformers. Which rule should combine the structured predictor with a flexible one? The linear opinion pool is the rule that commutes with marginalization [22]. Its log-likelihood is bounded by each component's (Proposition 4).

Foundation models as partners. The same rule combines a structured model with a *pretrained* foundation model. A foundation model is sharp where its vast prior fits; the lifted sites are exact where inputs are missing. Weights are chosen per episode by stacking [32] on a cross-fit of the episode's own labelled rows. No test label is used and nothing is retrained.

On four networks recorded in 2026, every pre-registered test of UPT $\oplus$ TabPFN passes (Table 1).

- **It improves TabPFN v2 on every network,** and more as inputs fail. It gains +0.17 [+0.14, +0.20] nats with all inputs and +0.60 [+0.49, +0.72] with six inputs missing (Figure 2).
- **UPT is the best partner.** It beats a plain transformer of equal size and training (+0.12 [+0.08, +0.16]) and the closed-form sites (+0.10 [+0.08, +0.12]).
- **It also improves TabPFN-3.5** (September 2026), which is far stronger than TabPFN v2: by +0.04 [+0.03, +0.06] nats with all inputs and +0.03 [+0.01, +0.04] with six missing.

Contributions.

1. **Theory (§3).** Products of sites are the free construction on sensors. Scalar experts fail under shared noise, while lifted sites are exact and minimal. The linear pool gives a per-query log-likelihood guarantee relative to each component.
2. **The Universal Pool Transformer (§4).** One transformer backbone over the cells of the support and the query feeds a lifted-site head, a free head and a gate, combined by the linear pool. At initialization UPT is the closed-form factor-analysis (FA) posterior up to 0.018 nats.
3. **Pooling with a foundation model (§4.1).** UPT $\oplus$ TabPFN applies the same pool, with per-episode cross-fit stacking.
4. **A pre-registered evaluation (§5).**
   - **Scale:** nine protocols, each committed before its data were scored, on thirteen networks in five domains.
   - **Fresh data:** Seven networks were recorded in 2026.
   - **Baselines:** TabPFN v2 and TabPFN-3.5, a matched-scale transformer, TNP-D, CNP, deep and classical kriging, gradient boosting and multiple imputation.
   - **Reporting:** every endpoint is reported, including the failures (Appendix B).

2 Problem setup

Episodes. An episode concerns one target sensor and its P input sensors in one time window.

- The support $\mathcal{D} = \{(x_i, m_i, y_i)\}_{i=1}^{n}$ holds $n = 48$ labelled rows. Reading x_{ij} is observed where $m_{ij} = 1$.
- A query is the reading x_S of an observed subset $S \subseteq \{1, \dots, P\}$.
- The model returns a predictive density $q(y \mid x_S, \mathcal{D})$.

Scoring. Methods are scored by the negative log-likelihood (NLL) of held-out targets, the strictly proper logarithmic score [10]. We also report the CRPS, the RMSE of the predictive mean, and the coverage of the central 90% interval.

Training and evaluation splits. Every learned model is trained on synthetic tasks, then fine-tuned on episodes from the network's *source* period with an identical recipe for every model. It is evaluated on fixed episodes from a later *test* period.

Sensor-loss conditions. In the condition "$+k$ dropped", k further observed inputs are removed from each query at random, on top of the natural missingness.

3 Theory: what missing sensors require

Write a Gaussian factor over a latent $z \in \mathbb{R}^L$ in natural parameters (Λ, η), where multiplying factors adds natural parameters. These pairs form a commutative monoid G. A site assignment $s : \{1, \dots, P\} \to G$ gives one factor per sensor. A mask $m \in \{0, 1\}^P$ lies in the free commutative monoid $\mathbb{N}^P$.

Proposition 1 (Products of sites are the free construction)**.** *For every site assignment s there is exactly one monoid homomorphism $F : \mathbb{N}^P \to G$ with $F(e_j) = s(j)$, namely $F(m) = \sum_j m_j s(j)$. Conversely, a predictor whose posterior natural parameters satisfy $F(A \sqcup B) = F(A) + F(B) - F(\emptyset)$ for disjoint masks is a product of the prior and per-sensor sites.*

Consequences. This is the universal property of the free commutative monoid [21].

- **Unseen masks.** Every mask, including masks never seen in training, follows from the P sites.
- **New sensors.** A new sensor extends F uniquely.
- **Limitation.** Cross-sensor interactions can enter only through the latent dimension, the Gaussian analogue of the limits of sum decompositions [29].

The next two results say how large that latent must be.

Proposition 2 (Scalar experts)**.** *Let $x_j = a_j y + d_j u + \sigma_j \epsilon_j$ with independent standard normal y, u, ϵ_j, $\sigma_j > 0$ and $a_j \neq 0$. Suppose each expert $t_j(y; x_j)$ is Gaussian in y and does not depend on S, and that $q(y \mid x_S) \propto p(y) \prod_{j \in S} t_j$ is Bayes for all singleton queries. Then q is Bayes for the pair $\{j, k\}$ if and only if $d_j d_k = 0$, and its weight on x_j can have the wrong sign.*

Lifted sites. Let $z = (y, h) \in \mathbb{R}^{1+K}$ with $h \sim \mathcal{N}(0, I)$. A lifted site is the rank-one factor of a linear-Gaussian measurement $x_j = w_j^\top z + c_j + e_j$, with $e_j \sim \mathcal{N}(0, \psi_j)$:

$$\Lambda_j = \frac{w_j w_j^\top}{\psi_j}, \qquad \eta_j = \frac{w_j (x_j - c_j)}{\psi_j}, \qquad q(z \mid x_S) = \mathcal{N}\Big(z; \Lambda^{-1}\eta, \Lambda^{-1}\Big), \quad (\Lambda, \eta) = (\Lambda_0, \eta_0) + \textstyle\sum_{j \in S} (\Lambda_j, \eta_j). \tag{1}$$

Proposition 3 (Exactness and minimal lifting)**.** *If the sensors follow a K-factor model with independent errors,* (1) *equals $p(z \mid x_S)$ for every S. Conversely, under a generic-rank condition, which holds for almost every loading matrix when $P \geq 2K + 3$, static sites over (y, h) with $h \in \mathbb{R}^{K'}$ reproduce $p(y \mid x_S)$ for all $|S| \leq 2$ only if $K' \geq K$.*

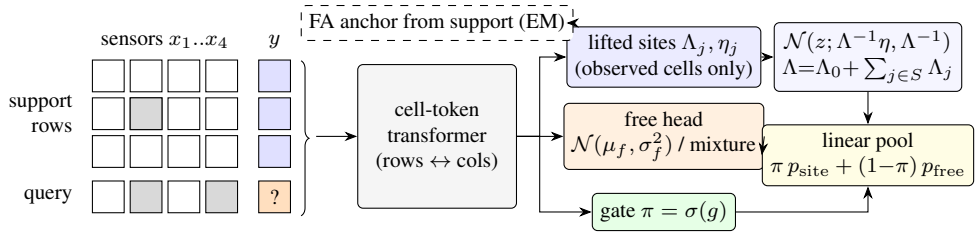


Figure 1: **The Universal Pool Transformer.** One transformer reads every cell of the support and the query (grey: missing). The query's observed cells emit corrections to their factor-analysis anchor sites, which combine exactly by adding natural parameters, so a missing sensor simply contributes nothing. The target token emits an unconstrained predictive and a gate, and the output is their linear opinion pool. With a zero site head the model is the closed-form FA posterior (Proposition 5).

Closed-form anchor. Fitting the factor model to the support by EM with missing entries [6, 26] gives a closed-form, mask-exact predictor, which we call *FA-Gaussian*. Proofs are in Appendix A.

Pooling with a flexible model. Real sensors are nonlinear and real noise is not exactly factor-structured, so a flexible model fits better. To combine the two, we use the linear opinion pool $p = \pi\, p_{\text{site}} + (1-\pi)\, p_{\text{free}}$. It is the pooling rule that commutes with marginalization (McConway, 1981) [22]: pooling joint predictives over z and then marginalizing the nuisance h gives the same result as marginalizing first. The logarithmic pool is instead the rule that commutes with Bayesian updating [9]. We tested it in protocol v5 and report it as an ablation.

Proposition 4 (Pool guarantee)**.** *For every query and outcome y,*

$$\text{NLL}_{\text{pool}}(y) \le \min\{\text{NLL}_{\text{site}}(y) - \log\pi,\ \text{NLL}_{\text{free}}(y) - \log(1-\pi)\}.$$

In particular, when the free component fails under sensor loss, the pool loses at most $-\log\pi$ relative to the lifted site, which handles any mask exactly.

4 The Universal Pool Transformer

Backbone. UPT uses a cell-token transformer over the support and the queries, in the style of TabPFN [16].

- **Tokens.** Every cell (row i, column j) becomes a token: the embedded value, or a learned *missing* embedding, plus a random feature embedding that distinguishes the columns.
- **Target column.** It carries the standardized support targets, and a learned query embedding in the query rows.
- **Attention.** Blocks alternate attention across the columns of a row and across the rows of a column. Query rows attend only to support rows.

Three heads on the query tokens.

1. **Lifted-site head.** Each observed query cell j outputs a correction $(\delta_j^{\text{off}}, \delta_j^{\text{slope}}, \delta_j^{D}, s_j)$ to its FA anchor site. The corrected site is the local linearization of sensor j's response at the anchor's prediction, and the sites combine by (1). Missing cells contribute nothing.
2. **Free head.** The target-column token outputs a Gaussian, or a small Gaussian mixture.
3. **Gate.** The same token outputs $\pi = \sigma(g)$. The predictive is the linear pool $\pi\, p_{\text{site}} + (1-\pi)\, p_{\text{free}}$.

Cost. The heads add $3d + 3$ parameters (about 0.1%), and inference is one backbone pass.

Proposition 5 (Initialization)**.** *With a zero site head and gate bias $+4$, the lifted site equals the FA-Gaussian, so for every query and outcome* $\text{NLL}_{\text{UPT}} \le \text{NLL}_{\text{FA}} - \log\sigma(4) = \text{NLL}_{\text{FA}} + 0.018$.

Scale-free target pipeline (SF). In-context transformers standardize the target internally. Our backbone inherited two fixed scales from its synthetic prior:

- a floor of 0.1 on the support standard deviation;
- a bound of ± 5 on the output log-variance.

The FA anchor also carries absolute ridges. On targets that vary little within an episode, these floors cap how sharp the predictive can be (§5.1). SF removes them:

- each episode's target is standardized by its support mean and standard deviation before the anchor fit and the network;
- the log-variance bound is widened to ± 10;
- predictions are mapped back to raw units ($\mathrm{NLL} + \log s$).

Because the inputs are already standardized per episode, the whole predictive becomes equivariant to affine changes of the target's units. We check this exactly in our test suite.

Training. We pretrain on synthetic factor-model tasks with nonlinear sensor responses. Query masks remove at most one sensor. Then we fine-tune on source-period episodes of each network.

- **Pretraining:** 30k steps for small models and 40k for large ones.
- **Fine-tuning:** 2,000 steps, with one recipe for every model.
- **UPT-L:** 2.1M parameters ($d = 128$, 8 layers).
- **Objective:** NLL, optionally plus the closed-form mixture CRPS [12], a sum of strictly proper scores.
- **Prediction:** we average 8 random feature embeddings by an equal-weight linear pool, the analogue of TabPFN's 8-member ensemble.
- **UPT-SF:** our final configuration. It is UPT-L with the SF pipeline, a three-component free mixture and the CRPS term, selected on development networks by a pre-registered rule.

4.1 Pooling with a foundation model

A pretrained foundation model F (here TabPFN v2 or TabPFN-3.5) and UPT each give a predictive for every query. We pool them with the same rule as inside UPT:

$$p(y \mid x_S, \mathcal{D}) = w\, p_{\mathrm{UPT}}(y \mid x_S, \mathcal{D}) + (1 - w)\, p_F(y \mid x_S, \mathcal{D}).$$

The weight w is chosen per episode by stacking [32], without test labels:

1. Split the support into halves A and B.
2. Fit both models in context on A and predict B, then the reverse. The held-out rows' inputs carry the same extra missingness as the test condition.
3. Choose $w \in \{0.02, 0.04, \ldots, 0.98\}$ to maximize the summed log density of the pool on the 48 held-out rows.

By Proposition 4, the pool can lose at most $-\log(1 - w)$ nats to F and $-\log w$ nats to UPT on any query. Stacking concentrates w where one of them is reliably better. Nothing is trained, and the cost is two extra in-context calls of each model per episode.

5 Experiments

Pre-registration. We ran nine protocols (v5–v13). Each was committed to a public repository before any score on its test data existed. Each fixes:

- networks, splits and seeded test panels;
- model recipes;
- endpoints, statistics and pass rules.

Every endpoint is reported, failures included (Table 4). Model changes were selected on data already used by an earlier protocol, by rules written into the next protocol, and then confirmed on networks never used before (v9, v10).

Networks. The thirteen networks are listed in Table 5.

Table 1: **Pre-registered endpoints on the four fresh 2026 networks.** NLL gain in nats (baseline − ours; positive favours the first method), paired 95% interval clustered by network-window; ✓: pass (gain ≥ 0.01 and lower bound > 0). Protocols v12 and v13 (TabPFN-3.5, September 2026) test the mechanism and a newer foundation model on the same networks after v11.

Protocol	Comparison	All inputs	3 dropped	6 dropped
v11	UPT-SF⊕TabPFN vs TabPFN v2	+0.17 [+0.14, +0.20] ✓	+0.24 [+0.20, +0.29] ✓	+0.60 [+0.49, +0.72] ✓
v11	UPT-SF⊕TabPFN vs UPT-SF alone	+0.91 [+0.76, +1.06] ✓	–	–
v13	UPT-SF⊕TabPFN-3.5 vs TabPFN-3.5	+0.04 [+0.03, +0.06] ✓	+0.03 [+0.01, +0.04] ✓	+0.03 [+0.01, +0.04] ✓
v13	UPT-SF⊕TabPFN-3.5 vs UPT-SF alone	+1.33 [+1.13, +1.53] ✓	–	–
v12	UPT-SF⊕TabPFN vs PFN-L⊕TabPFN	+0.12 [+0.08, +0.16] ✓	+0.12 [+0.08, +0.16] ✓	+0.09 [+0.05, +0.12] ✓
v12	UPT-SF⊕TabPFN vs FA⊕TabPFN	+0.10 [+0.08, +0.12] ✓	+0.11 [+0.10, +0.13] ✓	+0.12 [+0.10, +0.13] ✓
v10	UPT-SF alone vs TabPFN v2	-0.75 [-0.93, -0.58] ×	-0.63 [-0.81, -0.45] ×	-0.21 [-0.44, +0.03] ×

- **Traffic:** speed on METR-LA and PEMS-BAY, flow on PeMS03 and PeMS08.
- **Indoor climate:** Intel Lab temperature.
- **Air quality:** $PM_{2.5}$ in China and the UK; NO_2 in the UK.
- **Weather:** temperature and pressure from the German weather service.
- **Hydrology:** river discharge from three US states.

Seven networks have test periods in April–September 2026, after the release of TabPFN v2 (January 2025). TabPFN-3.5 was released on September 9, 2026, near the end of these periods. We cannot rule out that its training data include some of these public records; that would favour TabPFN-3.5, not our models.

Episodes and conditions. Each test episode predicts one held-out sensor from its 11 neighbours. It has 48 labelled support rows from the same time window, a day to a week. Metrics are computed on 48 query rows, under three conditions:

- the natural mask, "all sensors";
- $+3$ further inputs dropped at random;
- $+6$ further inputs dropped at random.

Baselines.

- **TabPFN v2** [16], in context with 8 estimators. Missing inputs are passed as NaN.
- **TabPFN-3.5** [18], the default model of the `tabpfn` package since September 2026, run exactly like TabPFN v2 (8 estimators, its default) on the four 2026 networks (protocol v13).
- **PFN-L.** A plain cell-token transformer with UPT's backbone, size and training budget, but only the free head: the matched-scale ablation. It is trained with the same SF pipeline and test-time ensemble as UPT.
- **Neural processes:** TNP-D [25] and CNP [8], trained on the same prior and fine-tuned with the same recipe.
- **Deep kriging:** an IGNNK-style inductive kriging model [31] trained on each network's source period. Its *calibrated* variant is recalibrated by an affine map on each episode's support, the stronger of the two.
- **Closed forms:** FA-Gaussian (UPT's anchor) and EM-Gaussian (simple kriging).
- **Tabular baselines:** LightGBM and multiple imputation (Appendix E).

UPT-SF is UPT-L with the SF pipeline, the CRPS-augmented objective and the 8-member test-time ensemble, chosen by the pre-registered rule of protocol v10.

5.1 Results

UPT ⊕ TabPFN on fresh 2026 data (protocols v11, v12). Every pre-registered endpoint of the pooled model passes (Table 1, pooled over 5,346 episodes on four networks).

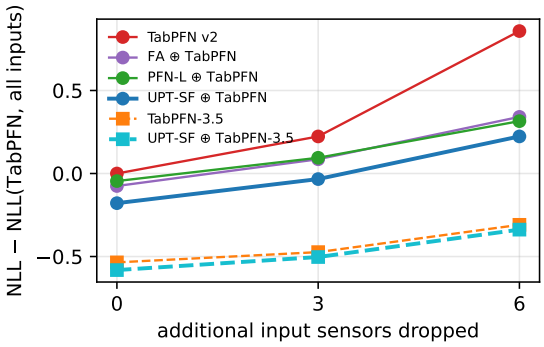


Figure 2: **Robustness to failing inputs** on the four 2026 networks. Mean NLL relative to TabPFN v2 with all inputs, as further input sensors are dropped from each query. TabPFN v2 degrades by 0.86 nats at six dropped inputs. Pooling it with any partner helps, and UPT is the best partner at every level. TabPFN-3.5 (dashed) is much stronger and more robust; pooling it with UPT still improves it at every level.

- **Against TabPFN v2,** the gain grows with the number of failed inputs: +0.17 [+0.14, +0.20] nats with all inputs, +0.24 [+0.20, +0.29] with three dropped, +0.60 [+0.49, +0.72] with six dropped.
- **On each network separately** the pool beats TabPFN v2 too: by 0.07–0.44 nats with all inputs and by 0.16–1.69 with six dropped (Table 2).
- **The stacking weights are interpretable.** With no test label, they place 72–83% of the weight on TabPFN for the two river networks with all inputs, whose near-deterministic targets suit its sharp predictive. They place 68–97% on UPT for pressure and $PM_{2.5}$, rising as inputs fail.

Is it the partner or just pooling? (protocol v12). Pooling TabPFN with any reasonable partner helps under input failure: TabPFN alone loses 0.86 nats at six dropped inputs (Figure 2). The partner still matters. With the same stacking rule, UPT is a better partner than:

- the plain transformer of equal size and training: +0.12 [+0.08, +0.16] nats with all inputs, +0.09 [+0.05, +0.12] with six dropped;
- the closed-form lifted sites: +0.10 [+0.08, +0.12] and +0.12 [+0.10, +0.13].

Both ingredients of UPT therefore contribute: the lifted sites beat the plain transformer, and the learning beats the closed form. The largest margin is on the pressure network: UPT $\oplus$ TabPFN scores -0.60 against -0.12 for PFN-L $\oplus$ TabPFN.

The newest foundation model (protocol v13). TabPFN-3.5 (September 2026) is far stronger than TabPFN v2 on these networks (by 0.3–1.3 nats) and degrades much less when inputs fail. The same pool, run unchanged, still improves it: +0.04 [+0.03, +0.06] nats with all inputs, +0.03 [+0.01, +0.04] and +0.03 [+0.01, +0.04] with three and six dropped (all pre-registered endpoints pass, Table 1). On pressure, UPT alone beats TabPFN-3.5 at every level (-0.57 vs -0.47 with all inputs) and the pool beats both (-0.64); a plain transformer of equal size is no help there (-0.47), and over the four networks UPT is the better partner by $+0.05$ to $+0.07$ nats. On the rivers, TabPFN-3.5 alone is better than the pool by 0.015–0.03 nats.

UPT alone across thirteen networks (protocols v6–v10). Table 3 reports every model on every network.

Against learned in-context baselines.

- At matched size and training, UPT has a lower mean NLL than the plain transformer on 12 of the 13 networks at every sensor-loss level. The exception is discussed below.

Table 2: **Pooling TabPFN v2 with a partner on four fresh 2026 networks** (mean test NLL, lower is better, on the episodes where TabPFN is defined). In parentheses: the mean stacking weight on the partner, chosen per episode by cross-fitting the episode's own labelled rows. Every pool uses the same rule; best per row in bold.

Network	Inputs	UPT-SF $\oplus$ TabPFN	PFN-L $\oplus$ TabPFN	FA $\oplus$ TabPFN	TabPFN v2
USGS NY (rivers)	all	**-3.699** (0.28)	-3.671 (0.28)	-3.674 (0.14)	-3.634
	-3	**-3.494** (0.36)	-3.466 (0.35)	-3.404 (0.25)	-3.401
	-6	**-3.146** (0.43)	-3.139 (0.40)	-2.979 (0.38)	-2.879
USGS WA (rivers)	all	-5.048 (0.17)	**-5.051** (0.18)	-5.042 (0.12)	-4.971
	-3	-4.871 (0.22)	**-4.877** (0.21)	-4.845 (0.17)	-4.789
	-6	-4.571 (0.27)	**-4.581** (0.23)	-4.523 (0.25)	-4.414
DWD pressure	all	**-0.595** (0.91)	-0.125 (0.88)	-0.332 (0.84)	-0.152
	-3	**-0.469** (0.95)	-0.012 (0.96)	-0.236 (0.93)	+0.211
	-6	**-0.228** (0.97)	+0.114 (0.96)	-0.091 (0.96)	+1.462
AURN $PM_{2.5}$	all	**-0.561** (0.68)	-0.525 (0.65)	-0.442 (0.41)	-0.432
	-3	**-0.490** (0.74)	-0.455 (0.69)	-0.361 (0.57)	-0.316
	-6	**-0.352** (0.79)	-0.322 (0.72)	-0.231 (0.73)	+0.074

Table 3: **Mean test NLL (nats; lower is better)** with all sensors (left of the slash) and with six extra sensors dropped (right). Learned models: mean over three seeds. Best per row in bold. Networks recorded in 2026 are below the rule; the last four are protocol v10, never used before its endpoints were scored. On USGS NY the UPT-SF mean is dominated by 15 episodes with a constant support target (§5.1).

Network	UPT-SF	PFN-L	TabPFN v2	TNP-D	Krig. (cal.)	FA-Gauss.
METR-LA	**-1.82/-1.60**	-1.48/-1.09	-1.79/-1.21	-1.37/-0.44	-0.50/+0.30	-0.15/-0.29
PEMS-BAY	**-2.68/-2.34**	-2.42/-2.03	-2.68/-2.06	-2.11/-0.92	-1.46/-0.57	-0.94/-1.02
Intel Lab	-0.95/**-0.02**	-0.32/+0.79	**-1.30**/+2.36	+0.03/+1.75	+0.94/+2.72	+0.49/+0.64
PeMS03	**-1.30/-1.00**	-1.03/-0.80	-1.04/-0.04	-0.93/-0.16	-0.42/-0.34	-0.57/-0.52
PeMS08	**-2.54/-2.12**	-1.12/-1.01	-1.82/-0.62	-1.07/-0.43	-0.68/-0.94	-1.28/-1.27
China air	**-0.25/-0.03**	-0.12/+0.06	-0.24/+0.92	-0.05/+0.21	+0.17/+0.51	+0.27/+0.26
DWD temp.	**+1.02/+1.22**	+1.04/+1.25	+1.32/+2.14	+1.19/+1.62	+1.39/+1.49	+1.52/+1.45
AURN NO_2	**+0.22/+0.41**	+0.23/+0.44	+0.30/+0.62	+0.31/+0.55	+0.37/+0.54	+0.47/+0.51
USGS PA	-2.62/-2.27	-2.26/-1.97	**-3.66/-2.72**	-1.82/-0.91	-0.82/-0.37	-0.86/-1.08
USGS NY	+2024.7/+1305.6	-1.56/-1.34	**-3.63/-2.88**	-1.41/-0.75	+85.3/+85.8	-0.17/-0.26
USGS WA	-3.31/-2.98	-2.73/-2.47	**-4.97/-4.41**	-2.51/-2.11	-1.95/-1.78	-2.22/-2.24
DWD pressure	**-0.57/-0.22**	-0.01/+0.14	-0.15/+1.46	+0.09/+1.40	+0.15/+0.65	+0.07/+0.03
AURN $PM_{2.5}$	-0.40/**-0.21**	-0.27/-0.10	**-0.43**/+0.07	-0.21/+0.06	-0.07/+0.08	+0.13/+0.08

- It beats TNP-D on all six v6/v7 networks at every level (pre-registered v8: $+0.38$, $+0.53$ and $+1.05$ nats at 0, 3 and 6 dropped inputs).
- It beats calibrated deep kriging by more than one nat (v8).

Against TabPFN v2 alone.

- With six inputs dropped, UPT is better on 10 of 13 networks, and better pooled over all thirteen (descriptive: $+0.33$ nats).
- With all inputs, it is better on 8 of 13.

Failures. We report all of them (Appendix B).

1. **UPT alone loses to TabPFN v2 on the river networks** (v9, v10). The pre-registered pooled NLL tests of UPT alone against TabPFN on the 2026 networks fail. On rivers TabPFN's predictive is far sharper, at about the same RMSE; UPT's synthetic prior (five sensors, one factor) is far narrower than TabPFN's. Pooling closes the gap with TabPFN v2, but not with TabPFN-3.5, which stays 0.015–0.03 nats ahead of the pool on rivers.

2. **A degenerate case of the scale-free pipeline.** When all 48 support targets of an episode are identical (15 of 1,437 New York episodes; TabPFN is undefined there), UPT's predictive collapses and its NLL explodes. This breaks the v10 comparisons of UPT alone against the matched transformer, TNP-D and kriging, which UPT wins on the other three networks. The fix is untested.
3. **With all inputs, TabPFN v2 alone remains better on 5 of 13 networks** (v6 P3, v9).

Ablations from earlier protocols (one network vs a pool of two, scale, float32 sites) are in Appendix B.

6 Related work

Kriging and virtual sensing. Graph neural kriging trains on a network's history and transfers to unseen nodes, for example IGNNK [31], GRIN [5] and spatio-temporal neural processes [17]. We compare with an IGNNK-style model, recalibrated in context on each episode's support. In-context models avoid retraining under drift.

Neural processes and prior-fitted networks. CNP [8], TNP [25], PFNs [24] and TabPFN v2 [16] learn to predict from a context set. They handle missing inputs implicitly. UPT adds an explicit structured component that handles missing inputs exactly.

Products of experts and opinion pools. Products of experts handle missing modalities by omission [15, 30]. Generalized products [3] and power EP [23] temper experts. The axiomatic theory of opinion pools [22, 9] characterizes the linear and logarithmic pools.

Learned message passing. Learned EP [14] and unrolled inference [11] amortize factor updates. Our lifted sites read their corrections from transformer tokens.

Missing values in supervised learning. NeuMiss [19] unrolls the Bayes predictor of a Gaussian model, but it is trained per dataset.

7 Limitations

Beyond the failures reported in §5.1:

- **Cost.** UPT $\oplus$ TabPFN runs both models, plus two half-support cross-fit calls per episode for the stacking weight.
- **Baselines.** TabPFN v2 (all networks) and TabPFN-3.5 (the 2026 networks) run in context without fine-tuning; neither is given the training history that UPT is fine-tuned on. Our kriging baseline is a local-graph adaptation, not a full-graph temporal model (GRIN [5], graph neural processes [17]), which may do better with a long, stationary history.
- **Data.** Episodes ignore temporal order within a window. The 2025–2026 data are provisional; we release content digests and download code. Protocols v12 and v13 reuse the v10 networks, and TabPFN-3.5 was released at the end of their test periods, so its training data may overlap with them.

8 Conclusion

Missing sensors call for structure, and the structure is unique: products of per-sensor sites are the free construction on sensors, lifting them to a target-plus-nuisance latent makes them exact under shared noise, and the linear opinion pool combines them with flexible models without losing what either does well. The Universal Pool Transformer realizes this structure in one network, and the same pool turns a pretrained foundation model into a predictor that is sharp when sensors are present and robust when they fail. Under protocols committed before their data were scored, UPT $\oplus$ TabPFN improved TabPFN v2 on every one of four networks recorded in 2026, and also improved the much stronger TabPFN-3.5; UPT was the best partner for both. A richer pretraining prior would close the remaining gap of UPT alone on near-deterministic targets.

References

[1] T. W. Anderson and Herman Rubin. Statistical inference in factor analysis. In *Proceedings of the Third Berkeley Symposium on Mathematical Statistics and Probability*, volume 5, pages 111–150, 1956.

[2] Peter Bodik, Wei Hong, Carlos Guestrin, Sam Madden, Mark Paskin, and Romain Thibaux. Intel lab data. `https://db.csail.mit.edu/labdata/labdata.html`, 2004.

[3] Yanshuai Cao and David J. Fleet. Generalized product of experts for automatic and principled fusion of Gaussian process predictions. *arXiv preprint arXiv:1410.7827*, 2014.

[4] David C. Carslaw and Karl Ropkins. openair — an R package for air quality data analysis. *Environmental Modelling & Software*, 27–28:52–61, 2012.

[5] Andrea Cini, Ivan Marisca, and Cesare Alippi. Filling the Gaps: Multivariate time series imputation by graph neural networks. In *International Conference on Learning Representations*, 2022.

[6] Arthur P. Dempster, Nan M. Laird, and Donald B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B*, 39(1):1–22, 1977. doi: 10.1111/j.2517-6161.1977.tb01600.x.

[7] Deutscher Wetterdienst, Climate Data Center. Hourly station observations of air temperature and pressure for Germany, recent. `https://opendata.dwd.de/climate_environment/CDC/`, 2026. Accessed October 2026.

[8] Marta Garnelo, Dan Rosenbaum, Christopher Maddison, Tiago Ramalho, David Saxton, Murray Shanahan, Yee Whye Teh, Danilo Rezende, and S. M. Ali Eslami. Conditional neural processes. In *Proceedings of the 35th International Conference on Machine Learning*, pages 1704–1713, 2018.

[9] Christian Genest. A characterization theorem for externally Bayesian groups. *The Annals of Statistics*, 12(3):1100–1105, 1984.

[10] Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.

[11] Karol Gregor and Yann LeCun. Learning fast approximations of sparse coding. In *Proceedings of the 27th International Conference on Machine Learning*, pages 399–406, 2010.

[12] Eric P. Grimit, Tilmann Gneiting, Veronica J. Berrocal, and Nicholas A. Johnson. The continuous ranked probability score for circular variables and its application to mesoscale forecast ensemble verification. *Quarterly Journal of the Royal Meteorological Society*, 132:2925–2942, 2006.

[13] Shengnan Guo, Youfang Lin, Ning Feng, Chao Song, and Huaiyu Wan. Attention based spatial-temporal graph convolutional networks for traffic flow forecasting. In *AAAI Conference on Artificial Intelligence*, 2019.

[14] Nicolas Heess, Daniel Tarlow, and John Winn. Learning to pass expectation propagation messages. In *Advances in Neural Information Processing Systems*, volume 26, 2013.

[15] Geoffrey E. Hinton. Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14(8):1771–1800, 2002.

[16] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637:319–326, 2025.

[17] Junfeng Hu, Yuxuan Liang, Zhencheng Fan, Hongyang Chen, Yu Zheng, and Roger Zimmermann. Graph neural processes for spatio-temporal extrapolation. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023.

[18] Benjamin Jäger, Nick Erickson, Léo Grinsztajn, Felix Birkel, Klemens Flöge, et al. TabPFN-3.5: Technical report, 2026. URL `https://arxiv.org/abs/2609.17895`.

[19] Marine Le Morvan, Julie Josse, Thomas Moreau, Erwan Scornet, and Gaël Varoquaux. NeuMiss networks: differentiable programming for supervised learning with missing values. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

[20] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. In *International Conference on Learning Representations*, 2018.

[21] Saunders Mac Lane. *Categories for the Working Mathematician*. Springer, 2 edition, 1998.

[22] Kevin J. McConway. Marginalization and linear opinion pools. *Journal of the American Statistical Association*, 76(374):410–414, 1981.

[23] Thomas Minka. Power EP. Technical Report MSR-TR-2004-149, Microsoft Research, 2004.

[24] Samuel Müller, Noah Hollmann, Sebastian Pineda Arango, Josif Grabocka, and Frank Hutter. Transformers can do Bayesian inference. In *International Conference on Learning Representations*, 2022.

[25] Tung Nguyen and Aditya Grover. Transformer neural processes: Uncertainty-aware meta learning via sequence modeling. In *International Conference on Machine Learning*, 2022.

[26] Donald B. Rubin and Dorothy T. Thayer. EM algorithms for ML factor analysis. *Psychometrika*, 47(1):69–76, 1982.

[27] Chao Song, Youfang Lin, Shengnan Guo, and Huaiyu Wan. Spatial-temporal synchronous graph convolutional networks: A new framework for spatial-temporal network data forecasting. In *AAAI Conference on Artificial Intelligence*, 2020.

[28] U.S. Geological Survey. National water information system: instantaneous values. `https://waterservices.usgs.gov/`, 2026. Accessed October 2026.

[29] Edward Wagstaff, Fabian B. Fuchs, Martin Engelcke, Michael A. Osborne, and Ingmar Posner. Universal approximation of functions on sets. *Journal of Machine Learning Research*, 23(151): 1–56, 2022.

[30] Mike Wu and Noah Goodman. Multimodal generative models for scalable weakly-supervised learning. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

[31] Yuankai Wu, Dingyi Zhuang, Aurelie Labbe, and Lijun Sun. Inductive graph neural networks for spatiotemporal kriging. In *AAAI Conference on Artificial Intelligence*, 2021.

[32] Yuling Yao, Aki Vehtari, Daniel Simpson, and Andrew Gelman. Using stacking to average Bayesian predictive distributions. *Bayesian Analysis*, 13(3):917–1007, 2018.

[33] Yu Zheng, Xiuwen Yi, Ming Li, Ruiyuan Li, Zhangqing Shan, Eric Chang, and Tianrui Li. Forecasting fine-grained air quality based on big data. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2015.

A Proofs

Proposition 1. $\mathbb{N}^P$ is the free commutative monoid on $\{1, \dots, P\}$: every m is uniquely $\sum_j m_j e_j$. A homomorphism F with $F(e_j) = s(j)$ must therefore satisfy $F(m) = \sum_j m_j F(e_j) = \sum_j m_j s(j)$, and this formula is a homomorphism, because addition in G is commutative and associative [21].

For the converse, let $F_0 = F(\emptyset)$ be the prior's natural parameters and set $s(j) = F(\{j\}) - F_0$. If $F(A \sqcup B) = F(A) + F(B) - F_0$ for all disjoint A, B, induction on $|S|$ gives $F(S) = F_0 + \sum_{j \in S} s(j)$. The posterior is thus the prior times one factor per observed sensor. □

Proposition 4. Both mixture components are non-negative. Hence $p_{\text{pool}}(y) \geq \pi\, p_{\text{site}}(y)$ and $p_{\text{pool}}(y) \geq (1 - \pi)\, p_{\text{free}}(y)$. Taking $-\log$ of each gives the two bounds.

Marginalization commutes with the linear pool by linearity of the integral: $\int (\pi p_1 + (1-\pi) p_2)\, dh = \pi \int p_1\, dh + (1-\pi) \int p_2\, dh$. McConway [22] shows that, under mild conditions, linear pools are the only pooling operators with this property. □

Proposition 5. With a zero site head, every corrected site equals its anchor site. The lifted posterior is then exactly the FA-Gaussian posterior of the anchor (Proposition 3 applied to the fitted factor model). With gate bias $+4$ and zero gate weights, $\pi = \sigma(4)$ for every query. Apply Proposition 4: $-\log \sigma(4) = \log(1 + e^{-4}) = 0.0180$. □

Scale equivariance of SF. Let $y' = ay + b$ with $a > 0$ be applied to the support and query targets of an episode. The SF pipeline standardizes by the support mean and standard deviation. These transform as $m' = am + b$ and $s' = as$ (the floor $10^{-4}(1 + |m|)$ is inactive whenever the support target varies). The standardized targets are therefore unchanged. The inputs do not depend on the target's units, so the network and the refitted anchor produce the same standardized predictive $\tilde{q}$. The raw-unit predictive is $q'(y') = \tilde{q}((y' - m')/s')/s'$, so $\text{NLL}' = \text{NLL} + \log a$, $\text{CRPS}' = a\,\text{CRPS}$ and the squared error scales by a^2. The test suite checks this for $a = 10^{-3}$ to within 2×10^{-3} nats, including the 8-member test-time ensemble. □

Proposition 2. Under the model, $p(x_j \mid y) = \mathcal{N}(x_j; a_j y, c_j)$ with $c_j = d_j^2 + \sigma_j^2$.

If $q(y \mid x_j) = p(y \mid x_j)$ for all x_j, then $p(y)\, t_j(y; x_j) \propto p(y)\, p(x_j \mid y)$ as functions of y. Hence $t_j(y; x_j) = \kappa_j(x_j)\, p(x_j \mid y)$, which is Gaussian in y with precision $a_j^2/c_j > 0$. For a pair, q is therefore the naive-Bayes posterior $p(y)\, p(x_j \mid y)\, p(x_k \mid y)$. Its posterior mean is linear in (x_j, x_k) with coefficients

$$(a_j/c_j,\ a_k/c_k) \,/\, \pi_q, \qquad \pi_q = 1 + a_j^2/c_j + a_k^2/c_k,$$

so the coefficient of x_j has the sign of a_j.

The Bayes posterior uses $C = \text{Cov}(x_j, x_k \mid y)$, with off-diagonal entry $d_j d_k$. Its mean coefficients are $(C^{-1}a)^\top/\pi$, with $\pi = 1 + a^\top C^{-1} a$. Both posteriors are Gaussian, so they coincide for all (x_j, x_k) if and only if $C^{-1}a = \text{diag}(C)^{-1}a$. Equivalently $a = C\,\text{diag}(C)^{-1}a$, whose off-diagonal components are $d_j d_k\, a_k/c_k = 0$ and $d_j d_k\, a_j/c_j = 0$. With $a_j a_k \neq 0$ this holds if and only if $d_j d_k = 0$.

For the sign flip, take $a = (1, 1)$, $d = (1, 0.5)$ and $\sigma = (0.05, 0.05)$. Then the Bayes coefficient of x_1 is $+0.499$ alone and -0.959 in the pair. □

Proposition 3, exactness. Given $z = (y, h)$, the readings are independent: $x_j \mid z \sim \mathcal{N}(w_j^\top z + c_j, \psi_j)$. Hence

$$p(z \mid x_S) \propto p(z) \prod_{j \in S} \mathcal{N}(x_j; w_j^\top z + c_j, \psi_j).$$

In natural parameters each factor contributes $w_j w_j^\top/\psi_j$ to the precision and $w_j(x_j - c_j)/\psi_j$ to the shift, which is (1) with $o_j = x_j - c_j$. The prior is Gaussian, so the product is the Gaussian (1). This holds for every S because the factors of unobserved sensors are absent from both sides. Marginalizing h gives $p(y \mid x_S)$. □

Proposition 3, minimal lifting. *Step 1: the product is a latent-factor model.* By the computation in the proof of Proposition 3 read backwards, a product of sites (1) with $o_j = x_j - c_j$ is the exact posterior over z of the model M_q: $z \sim \mathcal{N}(\mu_z, \Sigma_z)$ and, independently given z, $x_j \sim \mathcal{N}(w_j^\top z + c_j, \psi_j)$. Its y-marginal is M_q's conditional $p_q(y \mid x_S)$, and M_q's sensor covariance is $\Sigma_q = W\Sigma_z W^\top + \mathrm{diag}(\psi)$ with $W \in \mathbb{R}^{P\times(1+K')}$, so $\Sigma_q - \mathrm{diag}(\psi)$ has rank at most $1 + K'$.

Step 2: conditionals on $|S| \le 2$ identify the covariance. Write $V = \mathrm{Var}(y)$, $c = \mathrm{Cov}(x, y)$ and $\Sigma = \mathrm{Cov}(x)$. The case $S = \emptyset$ gives V. For $S = \{j\}$ the conditional has slope $\beta_j = c_j/\Sigma_{jj}$ and variance $r_j = V - c_j\beta_j$, and $\beta_j \neq 0$ because $c_j = a_j \neq 0$. Hence $c_j = (V - r_j)/\beta_j$ and $\Sigma_{jj} = c_j/\beta_j$. For $S = \{j, k\}$ the slope vector $b \neq 0$ solves $\Sigma_{\{jk\}}b = (c_j, c_k)$, and whichever entry of b is non-zero determines Σ_{jk} from one row. Since M_q has the same conditionals, the same formulas apply to it, so $\Sigma_q = \Sigma = LL^\top + \mathrm{diag}(\sigma^2)$.

Step 3: fewer factors cannot reproduce Σ. Suppose toward contradiction $K' < K$. Let $r = 1 + K$, so $LL^\top$ has rank r. Suppose $\Sigma = G + \Psi'$ with Ψ' diagonal and $\mathrm{rank}(G) \le r$. Fix a row i and two disjoint row sets I_1, I_2 not containing i with L_{I_1}, L_{I_2} nonsingular. In the $(r+1)\times(r+1)$ submatrix of Σ with rows $\{i\} \cup I_1$ and columns $\{i\} \cup I_2$, every entry except (i, i) lies off the diagonal, so it is shared by G and $LL^\top$. Both G and $LL^\top$ have rank at most r, so the corresponding minors vanish. Expanding along entry (i, i), its cofactor is $\det(L_{I_1}L_{I_2}^\top) \neq 0$, so $G_{ii} = (LL^\top)_{ii}$. Hence $G = LL^\top$, which has rank r [1]. Applying this to $G = W\Sigma_z W^\top$ from Step 1 gives $1 + K' \ge r = 1 + K$.

For almost every L all $r \times r$ submatrices are nonsingular, and $P \ge 2r + 1 = 2K + 3$ leaves two disjoint sets of r rows after deleting any row. □

B Protocols, endpoints and amendments

Pre-registration. Every protocol was committed to the public repository before any score on its test data existed. The repository history is available to reviewers. Each protocol fixes:

- the networks, splits and test panels (seeded);
- the models and recipes;
- the endpoints, and the statistic and pass rule of each.

Amendments were committed before the scores they affect. Table 4 lists every endpoint and its outcome. The amendments were:

- **v6, amendments 2 and 3:** undefined closed-form and TabPFN scores on degenerate episodes are recorded as NaN.
- **v8:** a vectorized but distributionally identical kriging sampler.
- **v9, amendment 1:** PFN-L was retrained, because the v4 checkpoints had not been kept.
- **v10, amendment 1:** a time-zone bug in the USGS parser was fixed before any panel was built.

Statistics. Endpoints compare per-episode metrics:

- learned models are averaged over three seeds;
- episodes are pooled over networks;
- paired 95% intervals are cluster-robust over network-time-windows;
- NLL endpoints pass when the gain is at least 0.01 nats and the lower bound is above zero.

C Networks and episodes

Construction. Every network is processed identically:

1. **Inputs:** the target sensor's $P = 11$ inputs are its nearest sensors. For traffic flow and river discharge, they are the sensors most correlated with it over the source period.
2. **Episodes:** each episode is one window of one target. It holds 48 support and 48 query rows, drawn from the rows where the target is observed.

Table 4: **Every pre-registered endpoint** of the protocols that evaluate UPT. Gain = baseline − UPT (positive favours UPT), paired 95% interval clustered by network-window.

Protocol	Endpoint	Gain	95% CI	Result
v6: METR-LA, PEMS-BAY, Intel				
	UPT vs transformer (PFN), NLL	+0.170	[+0.136, +0.205]	pass
	UPT non-inferior to 2-model pool	+0.126	[+0.091, +0.161]	pass
	UPT vs TabPFN v2, NLL, all sensors	-0.255	[-0.315, -0.195]	**fail**
	UPT vs PFN, synthetic, 16 sensors	+0.156	[+0.136, +0.175]	pass
v7: PeMS03, PeMS08, China air				
	UPT vs TabPFN v2, NLL, +3 dropped	+0.320	[+0.302, +0.337]	pass
	UPT vs TabPFN v2, NLL, +6 dropped	+0.989	[+0.945, +1.033]	pass
	UPT vs transformer (PFN), NLL	+0.361	[+0.283, +0.439]	pass
	float32 sites non-inferior to float64	+0.000	[-0.000, +0.000]	pass
	(sec.) UPT-L vs TabPFN v2, all sensors	+0.313	[+0.253, +0.372]	pass
	(sec.) UPT-L vs TabPFN v2, +3	+0.505	[+0.450, +0.561]	pass
	(sec.) UPT-L vs TabPFN v2, +6	+1.073	[+1.017, +1.128]	pass
	(sec.) UPT-L vs UPT	+0.230	[+0.174, +0.286]	pass
v8: the six networks above				
	UPT vs TNP-D, NLL, all sensors	+0.382	[+0.349, +0.415]	pass
	UPT vs CNP, NLL, all sensors	+1.521	[-0.252, +3.294]	**fail**
	UPT vs calibrated kriging, NLL, all sensors	+1.014	[+0.872, +1.156]	pass
	UPT vs TNP-D, NLL, +3 dropped	+0.526	[+0.487, +0.566]	pass
	UPT vs CNP, NLL, +3 dropped	+0.782	[+0.534, +1.029]	pass
	UPT vs calibrated kriging, NLL, +3 dropped	+1.037	[+0.889, +1.184]	pass
	UPT vs TNP-D, NLL, +6 dropped	+1.046	[+0.960, +1.132]	pass
	UPT vs CNP, NLL, +6 dropped	+1.004	[+0.740, +1.268]	pass
	UPT vs calibrated kriging, NLL, +6 dropped	+1.324	[+1.137, +1.511]	pass
v9: DWD temp., AURN NO_2, USGS PA				
	UPT* vs TabPFN v2, NLL, all sensors	-0.374	[-0.543, -0.205]	**fail**
	UPT* vs TabPFN v2, NLL, +3	-0.284	[-0.440, -0.127]	**fail**
	UPT* vs TabPFN v2, NLL, +6	+0.077	[-0.088, +0.243]	**fail**
	UPT* vs PFN-L (matched scale), NLL	+0.029	[+0.011, +0.047]	pass
	UPT* vs TNP-D, NLL	+0.258	[+0.194, +0.321]	pass
	UPT* vs calibrated kriging, NLL	+0.708	[+0.552, +0.863]	pass
	UPT* vs TabPFN v2, CRPS$/\sigma$	+0.012	[+0.008, +0.016]	pass
v10: USGS NY, USGS WA, DWD pressure, AURN $PM_{2.5}$				
	UPT-SF vs TabPFN v2, NLL, all sensors	-0.753	[-0.927, -0.579]	**fail**
	UPT-SF vs TabPFN v2, NLL, +3	-0.631	[-0.813, -0.449]	**fail**
	UPT-SF vs TabPFN v2, NLL, +6	-0.206	[-0.442, +0.029]	**fail**
	UPT-SF vs PFN-L (matched scale), NLL	-541.299	[-1603.170, +520.572]	**fail**
	UPT-SF vs TabPFN v2, CRPS$/\sigma$	+0.002	[-0.001, +0.005]	**fail**
	UPT-SF vs TNP-D, NLL	-541.163	[-1603.032, +520.706]	**fail**
	UPT-SF vs calibrated kriging, NLL	-517.787	[-1534.617, +499.043]	**fail**

Table 5: **The thirteen sensor networks.** Test period year; protocol that introduced the network; test episodes and network-time-window clusters.

Network	Quantity	Source	Test	Protocol	Episodes	Windows
METR-LA	traffic speed	Los Angeles loop detectors [20]	2012	v6	–	–
PEMS-BAY	traffic speed	Bay Area loop detectors [20]	2017	v6	–	–
Intel Lab	indoor temperature	Intel Berkeley lab motes [2]	2004	v6	–	–
PeMS03	traffic flow	Sacramento detectors [27]	2018	v7	–	–
PeMS08	traffic flow	San Bernardino detectors [13]	2016	v7	–	–
China air	$PM_{2.5}$	400 stations outside Beijing [33]	2015	v7	–	–
DWD temp.	air temperature	German weather service stations [7]	2026	v9	–	–
AURN NO_2	NO_2	UK air-quality network [4]	2026	v9	–	–
USGS PA	river discharge	Pennsylvania stream gauges [28]	2026	v9	–	–
USGS NY	river discharge	New York stream gauges [28]	2026	v10	–	–
USGS WA	river discharge	Washington stream gauges [28]	2026	v10	–	–
DWD pressure	air pressure	German weather service stations [7]	2026	v10	–	–
AURN $PM_{2.5}$	$PM_{2.5}$	UK air-quality network [4]	2026	v10	–	–

3. **Support dropout:** networks that are nearly complete receive 20% simulated per-sensor dropout on the support rows.
4. **Test targets:** these are a seeded sample of sensors.
5. **Splits:** the source and test periods are consecutive and do not overlap.

Snapshots. The 2025–2026 data are provisional: publishers may revise them. We therefore record the SHA-256 of each snapshot.

D Implementation

Architecture. Small models use $d = 64$, 4 layers, 4 heads and feed-forward width 128 (about 0.2M parameters). The large models UPT-L and PFN-L use $d = 128$, 8 layers, 8 heads and feed-forward width 512 (2.1M parameters).

Prior. Synthetic tasks draw $P = 5$ sensors from a one-factor model with nonlinear responses $x_j = a_j y + b_j + 0.4 \sin(f_j y) + d_j u + s_j \epsilon_j$. Support missingness is random.

Pretraining. AdamW with cosine decay.

- Small models: 30k steps, 16 tasks $\times$ 16 queries, lr 10^{-3}.
- Large models: 40k steps, 32 tasks $\times$ 16 queries, lr 5×10^{-4}.

Fine-tuning. Identical for every model and network:

- 4,000 source episodes and 2,000 AdamW steps;
- lr 3×10^{-4} with cosine decay;
- 64 episodes $\times$ 8 queries per step;
- query masks are the natural mask with at most one extra dropped sensor.

Baselines.

- **TabPFN v2:** `tabpfn` package, ungated v2 weights, 8 estimators. Missing inputs are passed as NaN, and NLL is computed from its bar distribution in raw units.
- **TNP-D and CNP:** pretrained and fine-tuned with the same recipe as our models.
- **IGNNK-style kriging:** trained on the source period of each network. A per-episode affine recalibration on the support gives the calibrated variant.
- **Others:** LightGBM, MICE with Bayesian linear regression, and the closed forms (FA-Gaussian, EM-Gaussian, ridge and Bayesian linear regression), as defined in the repository.

Compute.

- Hardware: one A100 80GB, or an L4, on Colab.
- Fine-tuning UPT-L: about 15 minutes per network and seed.
- TabPFN v2: about 1 to 2 hours per network of 1,200–1,500 episodes $\times$ 3 conditions under shared load.

E Additional results

Table 6: Per-network metrics, all sensors: NLL / CRPS/σ / RMSE/σ / 90% coverage.

Network	UPT-SF	PFN-L	TabPFN v2	TNP-D	Krig. (cal.)
METR-LA	-1.82/0.183/0.561/0.91	-1.48/0.194/0.569/0.94	-1.79/–/0.615/–	-1.37/0.212/0.586/0.94	-0.50/0.229/0.569/0.90
PEMS-BAY	-2.68/0.092/0.303/0.89	-2.42/0.095/0.307/0.94	-2.68/–/0.403/–	-2.11/0.130/0.384/0.93	-1.46/0.156/0.390/0.91
Intel Lab	-0.95/0.058/0.259/0.87	-0.32/0.060/0.198/0.95	-1.30/–/0.177/–	+0.03/0.088/0.257/0.93	+0.94/0.241/0.533/0.90
PeMS03	-1.30/0.064/0.163/0.90	-1.03/0.066/0.159/0.94	-1.04/–/0.192/–	-0.93/0.072/0.174/0.93	-0.42/0.103/0.218/0.89
PeMS08	-2.54/0.087/0.236/0.92	-1.12/0.092/0.231/0.94	-1.82/–/0.281/–	-1.07/0.097/0.238/0.94	-0.68/0.125/0.295/0.86
China air	-0.25/0.216/0.473/0.88	-0.12/0.219/0.474/0.91	-0.24/–/0.483/–	-0.05/0.233/0.490/0.91	+0.17/0.277/0.552/0.89
DWD temp.	+1.02/0.101/0.205/0.89	+1.04/0.101/0.204/0.90	+1.32/0.130/0.254/0.87	+1.19/0.114/0.226/0.89	+1.39/0.135/0.261/0.90
AURN NO_2	+0.22/0.327/0.614/0.89	+0.23/0.326/0.612/0.90	+0.30/0.352/0.655/0.90	+0.31/0.349/0.647/0.90	+0.37/0.368/0.674/0.90
USGS PA	-2.62/0.090/0.281/0.89	-2.26/0.087/0.256/0.93	-3.66/0.082/0.273/0.91	-1.82/0.123/0.327/0.92	-0.82/0.298/0.610/0.93
USGS NY	+2024.70/0.144/0.490/0.88	-1.56/0.143/0.466/0.92	-3.63/0.131/0.450/0.91	-1.41/0.180/0.516/0.91	+85.31/0.320/0.693/0.93
USGS WA	-3.31/0.099/0.266/0.89	-2.73/0.097/0.250/0.91	-4.97/0.091/0.250/0.92	-2.51/0.124/0.309/0.91	-1.95/0.238/0.516/0.89
DWD pressure	-0.57/0.024/0.057/0.89	-0.01/0.030/0.057/0.98	-0.15/0.037/0.087/0.90	+0.09/0.036/0.072/0.96	+0.15/0.047/0.102/0.91
AURN $PM_{2.5}$	-0.40/0.246/0.530/0.89	-0.27/0.248/0.528/0.91	-0.43/0.270/0.564/0.88	-0.21/0.260/0.543/0.91	-0.07/0.285/0.572/0.90

Table 7: Per-network metrics, +3 sensors dropped: NLL / CRPS/σ / RMSE/σ / 90% coverage.

Network	UPT-SF	PFN-L	TabPFN v2	TNP-D	Krig. (cal.)
METR-LA	-1.75/0.198/0.593/0.91	-1.38/0.211/0.601/0.94	-1.55/–/0.709/–	-1.14/0.242/0.652/0.93	-0.24/0.251/0.613/0.88
PEMS-BAY	-2.56/0.105/0.338/0.90	-2.30/0.108/0.339/0.94	-2.46/–/0.466/–	-1.83/0.157/0.450/0.92	-1.25/0.171/0.424/0.89
Intel Lab	-0.64/0.086/0.300/0.88	+0.01/0.089/0.284/0.92	+0.82/–/0.540/–	+0.71/0.164/0.408/0.87	+1.34/0.280/0.598/0.83
PeMS03	-1.18/0.071/0.175/0.91	-0.94/0.072/0.172/0.94	-0.81/–/0.238/–	-0.75/0.086/0.199/0.92	-0.43/0.104/0.222/0.90
PeMS08	-2.42/0.091/0.242/0.93	-1.10/0.095/0.237/0.95	-1.50/–/0.287/–	-0.96/0.107/0.254/0.93	-1.04/0.122/0.294/0.90
China air	-0.17/0.232/0.499/0.89	-0.05/0.234/0.499/0.91	+0.23/–/0.629/–	+0.04/0.257/0.529/0.91	+0.28/0.299/0.588/0.87
DWD temp.	+1.09/0.108/0.217/0.89	+1.10/0.107/0.216/0.90	+1.43/0.147/0.294/0.89	+1.31/0.127/0.248/0.92	+1.42/0.138/0.266/0.89
AURN NO_2	+0.29/0.350/0.654/0.88	+0.31/0.349/0.652/0.89	+0.40/0.386/0.712/0.87	+0.40/0.379/0.697/0.89	+0.43/0.387/0.706/0.88
USGS PA	-2.50/0.101/0.309/0.90	-2.16/0.097/0.279/0.93	-3.36/0.116/0.352/0.92	-1.54/0.156/0.393/0.90	-0.70/0.330/0.660/0.89
USGS NY	+1632.54/0.154/0.506/0.89	-1.47/0.154/0.487/0.92	-3.40/0.161/0.501/0.92	-1.19/0.207/0.555/0.91	+85.44/0.353/0.738/0.89
USGS WA	-3.21/0.109/0.288/0.90	-2.65/0.107/0.275/0.91	-4.79/0.112/0.299/0.91	-2.37/0.144/0.349/0.90	-1.95/0.238/0.519/0.90
DWD pressure	-0.44/0.027/0.064/0.89	+0.03/0.033/0.064/0.97	+0.21/0.058/0.149/0.94	+0.31/0.048/0.100/0.92	+0.27/0.053/0.117/0.88
AURN $PM_{2.5}$	-0.33/0.260/0.549/0.89	-0.22/0.261/0.548/0.91	-0.32/0.298/0.609/0.87	-0.13/0.280/0.572/0.91	-0.03/0.293/0.585/0.89

Table 8: Per-network metrics, +6 sensors dropped: NLL / CRPS/σ / RMSE/σ / 90% coverage.

Network	UPT-SF	PFN-L	TabPFN v2	TNP-D	Krig. (cal.)
METR-LA	-1.60/0.230/0.669/0.90	-1.09/0.248/0.674/0.94	-1.21/–/0.827/–	-0.44/0.298/0.760/0.89	+0.30/0.292/0.711/0.84
PEMS-BAY	-2.34/0.135/0.422/0.90	-2.03/0.140/0.420/0.94	-2.06/–/0.608/–	-0.92/0.216/0.575/0.87	-0.57/0.205/0.509/0.86
Intel Lab	-0.02/0.150/0.436/0.85	+0.79/0.157/0.425/0.86	+2.36/–/0.813/–	+1.75/0.292/0.609/0.78	+2.72/0.357/0.716/0.73
PeMS03	-1.00/0.082/0.196/0.92	-0.80/0.083/0.193/0.94	-0.04/–/0.434/–	-0.16/0.142/0.302/0.84	-0.34/0.109/0.232/0.88
PeMS08	-2.12/0.097/0.255/0.92	-1.01/0.101/0.248/0.95	-0.62/–/0.460/–	-0.43/0.142/0.306/0.87	-0.94/0.130/0.304/0.87
China air	-0.03/0.263/0.551/0.89	+0.06/0.265/0.550/0.91	+0.92/–/0.806/–	+0.21/0.302/0.603/0.90	+0.51/0.340/0.659/0.82
DWD temp.	+1.22/0.121/0.240/0.89	+1.25/0.119/0.237/0.88	+2.14/0.265/0.495/0.72	+1.62/0.168/0.317/0.91	+1.49/0.145/0.278/0.87
AURN NO_2	+0.41/0.388/0.716/0.87	+0.44/0.388/0.717/0.87	+0.62/0.452/0.813/0.80	+0.55/0.431/0.781/0.85	+0.54/0.422/0.761/0.85
USGS PA	-2.27/0.125/0.360/0.91	-1.97/0.120/0.338/0.92	-2.72/0.202/0.508/0.85	-0.91/0.220/0.501/0.85	-0.37/0.402/0.772/0.81
USGS NY	+1305.58/0.176/0.537/0.89	-1.34/0.179/0.527/0.91	-2.88/0.236/0.609/0.86	-0.75/0.258/0.622/0.87	+85.80/0.436/0.859/0.81
USGS WA	-2.98/0.128/0.333/0.90	-2.47/0.129/0.328/0.90	-4.41/0.154/0.385/0.86	-2.11/0.177/0.412/0.88	-1.78/0.274/0.588/0.85
DWD pressure	-0.22/0.034/0.079/0.90	+0.14/0.039/0.080/0.96	+1.46/0.181/0.391/0.76	+1.40/0.100/0.206/0.76	+0.65/0.069/0.151/0.79
AURN $PM_{2.5}$	-0.21/0.285/0.587/0.89	-0.10/0.286/0.586/0.90	+0.07/0.383/0.728/0.77	+0.06/0.328/0.645/0.88	+0.08/0.315/0.617/0.86